

Læringsutbyttebeskrivelser for TDT4160

Sist oppdatert 15/08-25

Overordnede læringsutbyttebeskrivelser

Kunnskaper:

- **K1:** Studenten skal kjenne til datamaskiners konstruksjon og virkemåte, inkludert hvordan man oppnår høy effektivitet.
- **K2:** Studenten skal forstå grensesnittet mellom programvare og maskinvare.
- **K3:** Studenten skal forstå hvordan man bygger enkle og effektive prosessorer, inkludert enkeltsykel, flersykel, og samlebåndsarkitekturer.
- **K4:** Studenten skal forstå prinsippene bak hvordan man bygger effektive minnesystemer, inkludert hurtigbuffer og virtuelt minne.
- **K5:** Studenten skal forstå hvordan abstraksjon og struktur benyttes for å oppnå høy effektivitet og til å håndtere kompleksitet i datamaskiner.

Ferdigheter:

- **F1:** Studenten skal være i stand til å formulere enkle programmer i assemblykode.
- **F2:** Studenten skal være i stand til å lese blokkdiagrammer.
- **F3:** Studenten skal kunne relatere blokkdiagrammer på ulike abstraksjonsnivå til hverandre.

Generell kompetanse:

- **G1:** Studenten skal forstå den generelle virkemåten til en datamaskin og kunne anvende denne kunnskapen i prosjekter på alle abstraksjonsnivå.

Merk: Læringsutbyttebeskrivelsene er ikke identisk med læringsutbyttebeskrivelsene på fagets offisielle nettside, men endringene er kun kosmetiske.

Spesifikke læringsutbyttebeskrivelser

Merk: Tilordningen av læringsutbyttebeskrivelser til forelesninger er fra i fjor, og den vil bli litt annerledes i år. Jeg oppdaterer dette etter hvert som jeg ser hvor mye jeg kommer gjennom i hver forelesning. Det vil også bli endringer på enkelte læringsutbyttebeskrivelser.

Tema	Læringsutbytte	Undervisnings- materiell	Oppgaver i boka	Eksamens- oppgaver
T1: Introduksjon og ytelse [K1, K5, G1]				
T1.1: Datamaskintyper og de 7 store ideene	• Studenten kjenner til de 7 datamaskintypene og deres viktigste egenskaper. • Studenten kjenner til de 7 store ideene i datamaskinarkitektur. • Studenten kan beskrive størrelser med rett prefiks (for eksempel GB og GiB).	• Forelesning 1 • Kapittel 1.1 og 1.2	• 1.1 • 1.2	<i>Ingen spesielt relevante¹</i>
T1.2: Under overflaten	• Studenten kjenner rollen til applikasjonsprogramvare og systemprogramvare, herunder operativsystemet og kompilatoren. • Studenten kan beskrive de fem hovedkomponentene i en datamaskin. • Studenten kan forklare prinsippet om lagrede program. • Studenten kan gjengi produksjonsprosessen for integrerte kretser.	• Forelesning 1 • Kapittel 1.3 til 1.5	• 1.3 • 1.11	<i>Ingen spesielt relevante</i>
T1.3: Ytelse	• Studenten kjenner til de viktigste ytelsesmetrikkene i datamaskinarkitektur. • Studenten forklare forskjellen på kjøretid og båndbredde og velge den mest hensiktsmessige for gitte arkitekturoppgaver.	• Forelesning 1 og 2 • Kapittel 1.6, 1.7, 1.8, 1.9	• 1.6 • 1.7 • 1.9	• Ord. 24: Oppg. 5.4

¹ «Ingen spesielt relevante» betyr at jeg ikke har funnet noen gamle eksamensoppgaver som jeg mener gir vesentlig trening i dette temaet. Det betyr ikke at det ikke eksisterer oppgaver som er innom temaet. For eksempel er noen av disse temaene dekket av korte faktaspørsmål (typisk oppgave 1 i noen eksamener).

	• Studenten kan forklare «The Iron Law» og kan bruke den til å forutsi hvordan endringer i arkitekturen påvirker kjøretid. • Studenten kjenner til hvordan spenning og klokkefrekvens påvirker effektforbruk og strømforbruk. • Studenten vet hva en testprogramsamling er og hvorfor de brukes.			
T1.4: Parallele datamaskiner	• Studenten vet hvorfor stort sett alle datamaskiner i dag har mer enn én prosessorkjerne. • Studenten kan bruke Amdahl's lov til å analysere ytelsesforbedring i parallelle systemer.	• Forelesning 2 • Kapittel 1.8 og 1.11	• 1.10 • 1.16	<i>Ingen spesielt relevante</i>
T2: Introduksjonssett [K1, K2, K5, F1, G1]				
T2.1: Instruksjoner	• Studenten kjenner til de tre designprinsippene for instruksjonssettdesign og deres motivasjon. • Studenten kan oversette fra et høynivåspråk til assemblyinstruksjoner (og omvendt). • Studenten kan oversette fra assemblyinstruksjoner til maskinkode (og omvendt). • Studenten kjenner til instruksjonsformatene i RISC-V og kan forklare hvorfor de er definert slik de er. • Studenten forstår hvordan instruksjoner lagres i minnet og hvordan dette utnyttes til å implementere kontrollflyt.	• Forelesning 2 og 3 • Kapittel 2.1 til 2.7	• 2.1 • 2.2 • 2.3 • 2.4 • 2.11 • 2.12 • 2.13	• Kont 25: Oppgave 1 • Ord. 24: Oppgave 1 • Kont 24: Oppgave 1 • Ord. 22: Oppg. 11²
T2.2: Funksjonskall	• Studenten vet hvilke oppgaver som skal utføres ved et funksjonskall og kan forklare konseptet kallkonvensjon. • Studenten kjenner RISC-V minnekartet (kan gjengi og forklare figur 2.13). • Studenten forstår forskjellen på statiske og dynamiske data.	• Forelesning 3 • Kapittel 2.8	• 2.30 • 2.31 • 2.32	<i>Ingen spesielt relevante</i>

² Oppgavetypen er relevant, men den ville vært vanskeligere nå enn da den ble gitt siden vi bruker RISC-V ISA i øvingene. Vedlegget gir imidlertid nok informasjon til at oppgaven skal kunne besvares.

T2.3: Instruksjoner, diverse	• Studenten kjenner til hvordan tekst representeres i en datamaskin. • Studenten skal vite hvordan vi håndterer store konstanter og (unngår) lange hopp (PC-relativ adressering). • Studenten skal kunne forklare RISC-Vs fire adressemodi. • Studenten skal kunne forklare skrittene involvert i oversettelse og oppstart av programmer (kunne gjengi og forklare figur 2.20)	• Forelesning 3 • Kapittel 2.9 til 2.10 og 2.12	• 2.22 • 2.34	<i>Ingen spesielt relevante</i>
T3: Enkeltsykelprosessor (inkl. kombinatorisk logikk og aritmetisk-logisk enhet) [K1, K2, K3, K5, F2, F3, G1]				
T3.1: Enkeltsykelprosessor	• Studenten skal kunne forklare begrepene datasti og kontrollenhet. • Studenten skal kunne forklare mikroarkitekturen til enkeltsykelprosessoren (kunne gjengi og forklare figur 4.21). • Studenten skal kunne sette opp rett kontrollord for en instruksjon, inkludert korrekt bruk av «don't care». • Studenten skal kunne identifisere instruksjonstype fra et kontrollord.	• Forelesning 4 og 5 • Kapittel 4.1 til 4.4	• 4.1 • 4.3 • 4.5 • 4.7	• Kont 25: Oppgave 2 • Ord. 24: Oppgave 2 • Ord. 23: Oppgave 5 • Kont 23: Oppgave 3
T3.2: Kombinatorisk logikk	• Studenten skal kjenne til de logiske portene AND, OR, og inverter og deres symboler samt kunne beskrive oppførselen deres med sannhetstabeller. • Studenten skal kjenne til symbolene og kunne gjengi sannhetstabellene for kombinerte logiske porter (NAND og NOR) og mer avanserte enheter (multiplekser og dekodere). • Studenten skal kunne oversette mellom boolske uttrykk på sum-av-produkt form og sannhetstabeller. • Studenten skal kjenne til konseptet buss og notasjonen for adressering av enkeltlinjer i busser.	• Forelesning 4 • Kapittel A-1 til A-3	• A.5 • A.7 • A.8 • A.10 • A.11	• Kont 24: Oppgave 2 • Ord. 23: Oppgave 3 • Kont 23: Oppgave 2 • Ord. 22: Oppgave 4 og 5 • Kont 22: Oppgave 3

T3.3: Aritmetisk-logisk enhet	• Studenten skal kunne konstruere en 32-bit aritmetisk-logisk enhet (ALU) som kan gjøre addisjon og subtraksjon samt logisk AND og OR og nulldeteksjon (gjengi og forklare figurene A.5.7 og A.5.8). • Studenten skal kunne utføre multiplikasjon og divisjon av binære tall og beskrive hvordan disse implementeres i maskinvare. • Studenten skal kunne oppgi fordeler og ulemper med å representere tall i et «fixed-point» format. • Studenten skal kunne motivere for hvorfor vi trenger flyttall og beskrive prinsippene for hvordan addisjon, subtraksjon, multiplikasjon, og divisjon med flyttallsverdier utføres i maskinvare. • Studenten vite hva overflyt er og forstå når det oppstår. • Studenten skal kjenne til hvordan man implementerer SIMD-instruksjoner og hva de brukes til.	• Forelesning 5 og 6 • Kapittel 3.1 til 3.6 og A-5	• 3.3 • 3.6 til 3.8 • 3.12 • 3.18 • 3.20 til 3.23	• Ord. 22: Oppgave 6 og 7
T4: Flersykelprosessor (og sekvensiell logikk) [K1, K2, K3, K5, F2, F3, G1]				
T4.1: Flersykel-prosessor	• Studenten skal kunne forklare mikroarkitekturen til en flersykelprosessor (kunne gjengi og forklare figur e.4.5.4). • Studenten skal kunne forklare hvorfor kontrollenheten til flersykelprosessoren blir en tilstandsmaskin. • Studenten skal kunne beskrive sammenhengen mellom kontrollordet til flersykelprosessoren og tilstanden i tilstandsmaskinen. • Studenten skal kunne beskrive fordeler og ulemper ved flersykelprosessoren sammenliknet med enkeltsykelprosessoren.	• Forelesning 6 og 7 • Kapittel 4.5	<i>Ingen tilgjengelige oppgaver</i>	• Ord. 24: Oppgave 3
T4.2: Sekvensiell logikk	• Studenten skal kunne beskrive oppførselen til komponentene SR-lås og D-vippe.	• Forelesning 6	• A.36	• Ord. 23: Oppgave 2

	• Studenten skal kunne konstruere registre og registerfiler med kombinatoriske og sekvensielle logiske komponenter (kunne gjengi og forklare figur A.8.8 og figur A.8.9). • Studenten skal kunne forklare hva en tilstandsmaskin er og beskrive hvordan den implementeres i maskinvare (kunne gjengi og forklare figur A.10.3). • Studenten skal kunne forklare funksjonen til klokkesignalet i en datamaskin og begrepet «kritisk sti».	• Kapittel A.8, A.10 og A.11	• A.37 • A.38 • A.39	• Ord. 22: Oppgave 2 og 3 • Kont 22: Oppgave 2
T5: Samlebåndsprosessorer [K1, K2, K3, K5, F2, F3, G1]				
T5.1: Samlebåndsprosessor med 5 steg	• Studenten skal kunne beskrive fordeler og ulemper ved samlebåndsprosessen sammenliknet med flersykelprosessen og enkeltsykelprosessen. • Studenten skal kunne forklare hvordan oppstartskostnad og balanse mellom steg i samlebåndet påvirker ytelse. • Studenten skal kunne beskrive forskjellen mellom avhengigheter og farer samt gjengi de tre hovedgruppene av farer (strukturfarer, datafarer, og kontrollfarer). • Studenten skal kjenne til de fire overordnede strategiene for å håndtere farer (unngåelse, videresending, stans, og prediksjon) • Studenten skal kunne forklare den grunnleggende mikroarkitekturen til 5-stegs samlebåndsprosessen (kunne gjengi og forklare figur 4.43). • Studenten skal kunne forklare hvordan kontroll implementeres i 5-stegs samlebåndsprosessen. • Studenten skal kunne forklare hvordan videresending og stans kan brukes til å håndtere datafarer samt kunne analysere hvordan strategiene påvirker prosessorens ytelse.	• Forelesning 7 og 8 • Kapittel 4.6 til 4.8	• 4.16 • 4.17 • 4.18 • 4.22-4.25 • 4.27 • 4.28.1 og 4.28.2	• Kont 25: Oppgave 3 • Ord. 24: Oppgave 4 • Kont 24: Oppgave 3 • Ord. 23: Oppgave 4 • Kont 23: Oppgave 4 • Ord. 22: Oppgave 10 (litt relevant) • Kont 22: Oppg. 10

	• Studenten skal kunne forklare hvordan kontrollfarer kan håndteres med stans og prediksjon samt analysere hvordan strategiene påvirker prosessorens ytelse.			
T5.2: Unntak og avbrudd	• Studenten skal kunne forklare begrepene unntak og avbrudd, inkludert presise unntak. • Studenten skal kunne beskrive hvordan presise unntak kan implementeres i 5-stegssamlebåndet. • Studenten skal kjenne til prinsippene bak hvordan presise unntak implementeres i prosessorer som utfører instruksjoner ut-av-rekkefølge.	• Forelesning 8 og 9 • Kapittel 4.10 og 4.11	• 4.30	• Ord. 22: Oppgave 10 (litt relevant)
T5.3: Prosessorer med høyere ytelse	• Studenten skal kunne beskrive hvordan en prosessor utnytter parallellitet i tid og parallellitet i rom. • Studenten skal kunne forklare begrepet «speculation». • Studenten skal kunne gjenkjenne og beskrive RAW, WAW og WAR farer og avhengighetene som skaper dem. • Studenten skal forstå prinsippene bak hvordan «register renaming» fjerner WAW og WAR farer. • Studenten skal kjenne til fordeler og ulemper med «static multi-issue» prosessorer. • Studenten skal kjenne til prinsippene bak hvordan prosessorer som utfører instruksjoner ut-av-rekkefølge implementeres («dynamic multi-issue»).	• Forelesning 8 og 9 • Kapittel 4.11	• 4.31	<i>Ingen spesielt relevante</i>
T6: Minnesystemet [K1, K2, K4, K5, F2, F3, G1]				
T6.1: Minnehierarki og hurtigbuffer	• Studenten skal kunne forklare begrepene lokalitet i tid og lokalitet i rom og vite hvorfor lokalitet oppstår. • Studenten skal forstå hvordan og hvorfor et hierarki av minner kan gi illusjonen av ett stort og raskt minne.	• Forelesning 9, 10 og 11 • Kapittel 5.1, 5.3 og 5.4	• 5.1 • 5.2 • 5.3 • 5.5 • 5.7	• Kont 25: Oppgave 4 • Ord. 24: Oppgave 5

	• Studenten skal forstå hvordan vi konstruerer et direktetilordnet hurtigbuffer (kunne gjengi og forklare figur 5.10) samt hvordan det kan utvides til å håndtere hurtigbufferblokker som består av mer enn ett ord. • Studenten kunne forklare hvordan hurtigbuffer kan integreres i en samlebåndsarkitektur. • Studentene skal kjenne til prinsippene for hvordan hurtigbuffer håndterer skriveoperasjoner. • Studenten skal kunne beregne hvordan minneaksessetid påvirker datamaskinens ytelse. • Studenten skal kunne beregne hvordan treffraten i flernivå hurtigbuffer påvirker den gjennomsnittlige aksessiden. • Studenten skal forstå hvorfor settassosiative og fullassosiative hurtigbuffer kan øke treffraten og hvordan de konstrueres (gjengi og forklar figur 5.18). • Studenten skal kjenne til hvordan man på programvarenivå kan påvirke treffraten i hurtigbuffer.		• 5.10 • 5.11	• Kont 24: Oppgave 4 • Ord. 23: Oppgave 6 • Ord. 22: Oppgave 8 • Kont 22: Oppgave 4 • Ord. 21: Oppgave 2 og 3 • Ord. 21: Oppgave 6
T6.2: Minne-teknologier	• Studenten skal kunne forklare begrepene volatilt og ikke-volatilt minne samt statisk og dynamisk minne. • Studenten skal kjenne til omtrentlig aksessetid og kostnad per bit for SRAM, DRAM, Flash, og magnetisk disk og kunne forklare hvordan dette påvirker hvordan teknologiene brukes i minnehierarkiet. • Studenten skal kunne konstruere et SRAM minne fra grunnleggende kombinatoriske og sekvensielle komponenter (kunne gjengi og forklare figur A.9.3). • Studenten skal kunne beskrive prinsippene for hvordan vi konstruerer DRAM minner (kunne gjengi og forklare figur A.9.5 og A.9.6).	• Forelesning 10 • Kapittel 5.2 og A.9	<i>Ingen tilgjengelige oppgaver</i>	<i>Ingen spesielt relevante</i>

T6.3: Virtuelt minne	• Studenten skal kunne forklare hvordan virtuelle adresser oversettes til fysiske adresser, inkludert arbeidsfordelingen mellom maskinvare og programvare. • Studenten skal kunne forklare hvordan et «Translation Lookaside Buffer (TLB)» brukes til å implementere rask adresseoversettelse. • Studenten skal kunne forklare hvordan virtuelt minne kan brukes til å beskytte prosesser fra hverandres minneaksesser. • Studenten skal kunne forklare hvordan sidefeil håndteres. • Studenten skal kunne forklare hva en virtuell maskin er.	• Forelesning 11 • Kapittel 5.6 og 5.7	• 5.16 • 5.19	• Ord. 24: Oppgave 5.5 • Kont 23: Oppgave 5 og 6 • Ord. 21: Oppgave 4
T7: Parallelle datamaskiner [K1, K5, G1]				
T7.1: Ytelse og Flynns taksonomi	• Studenten skal kunne forklare hvorfor det er enklere å utnytte parallellitet mellom uavhengige program enn innad i ett program. • Studenten skal kunne bruke Amdahls lov til å analysere skalerbarheten til parallelle programmer. • Studenten skal kjenne til konseptene sterk og svak skalering og kunne bruke dem til å vurdere skalerbarheten til ulike applikasjoner. • Studenten skal kunne forklare Flynns taksonomi og kunne gi eksempler på SISD, SIMD, og MIMD maskiner. • Studenten skal kunne bruke Roofline-modellen og konseptet operasjonsintensitet til å analysere hvorvidt en applikasjon er minnebundet eller beregningsbundet på en gitt datamaskin.	• Forelesning 11 og 12 • Kapittel 6.1 til 6.3 og 6.11	• 6.2 • 6.3 • 6.11	• Kont 25: Oppg. 5.1
T7.2: Multiprosessorer	• Studenten kan beskrive den overordnede arkitekturen til multiprosessorer med delt minne og distribuert minne samt kjenne til deres styrker og svakheter.	• Forelesning 11 og 12 • Kapittel 6.4, 6.5 og 6.8	• 6.6 • 6.7 • 6.19	• Kont 25: Oppg. 5.2

	• Studenten skal kunne beskrive den overordnede arkitekturen til en flerkjerneprosessor og kunne motivere for hvorfor arkitekturen er som den er. • Studenten skal kunne forklare hvorfor synkronisering er nødvendig og de grove trekkene i hvordan synkronisering implementeres. • Studenten skal kjenne til behovet for å holde hurtigbufferer koherente og de grove trekkene i hvordan det løses. • Studenten skal kunne beskrive minnekonsistensproblemet og kjenne til de grove trekkene i hvordan det løses.	• Kapittel 5.10 • Kapittel 2.11		
T7.3 Akseleratorer (Grafikkprosessorer og domene-spesifikke akseleratorer)	• Studenten skal vite hva en heterogen datamaskin er og kunne forklare hvorfor datamaskiner med høy ytelse typisk er heterogene. • Studenten skal kunne forklare den overordnede arkitekturen til grafikkprosessorer (GPUer) samt kunne beskrive programmeringsmodellen som benyttes når man bruker GPUer til generell beregning. • Studenten skal kunne beskrive likheter og forskjeller mellom GPUer, vektorprosessorer, og SIMD-instruksjoner. • Studenten skal kunne forklare hva en domene-spesifikk akselerator (DSE) er og motivere for hvorfor disse kan oppnå høyere ytelse enn mer generelle arkitekturer (altså CPUer og GPUer).	• Forelesning 12 • Kapittel 6.6, 6.7	<i>Ingen (fordi 6.14 er urimelig tidkrevende)</i>	• Kont 25: Oppgave 5.3 og 5.4 • Kont 24: Oppgave 5